

Julia and Python: Which Language is Quicker?



Julia is a dynamic programming language with a high level of abstraction. While it is a general-purpose language that may be used to develop any program, it has several characteristics that are ideally suited to numerical analysis and computational research. Python was designed in the early 1990s as a simple object-oriented programming language, but has evolved significantly since then. This article takes a deeper look at the performance of both for neural networks and machine learning.

Julia's architecture features parametric polymorphism in a dynamic programming language, as well as a multiple dispatch programming paradigm as its primary programming model. It allows concurrent, parallel, and distributed computing with or without message passing interface (MPI) or the built-in 'OpenMP-style' threads, as well as the direct invocation of C and FORTRAN libraries without intermediate code. Julia employs a just-in-time (JIT) compiler, which the Julia community refers to as 'just-ahead-of-time' (JAOT) since it compiles all code to machine code by default before running it.

Unlike Python, Julia was designed specifically for use in statistics and machine learning.

Julia can fly through linear algebra, whereas Python can plod through it. This is because Python was never designed to accommodate all of the matrices and equations that machine learning requires. Python isn't bad by any means, especially with NumPy, but Julia is a lot better tailored to this kind of mathematics in terms of a no-package experience. Julia's operand system is a lot more like R's than Python's, which is a significant plus. The majority of linear algebra can be completed in less time and with less effort.

As we know, in recent years Python has dominated the areas of machine learning and data science. Because of the variety of third-party libraries that we can use in Python, it helps to develop machine learning code easily. While there are so many advantages of Python, there is one major drawback — it's an interpreted language, which makes it slow. This is the age of data, and the more data there is the more time it takes to work on it. That's where Julia comes into the picture.

Most of the research work on Julia so far has been on topics like high power computing or the scientific